

Zomato Customer Analytics Case Study

Zomato is one of India's largest food delivery platforms, connecting millions of customers with restaurants across multiple cities.

Over the last few months, the leadership team has observed that while the number of registered customers continues to grow, overall business performance is not improving at the expected rate. Revenue growth has become inconsistent, customer retention appears to be declining, and certain restaurant partners are underperforming.

OBJECTIVE

Finding what holds Zomato's growth back

Registered customers keep growing, but revenue is uneven, retention is slipping and some restaurant partners underperform. This project analyses customer, restaurant and order data to find out why, and what to do about it.

Revenue

Customers

Restaurants

Coupons

Cancellations and refunds

Churn

Data: 4,999 customers · 50,000 orders · 200 restaurants · Jan 2024 to Apr 2026

TECH STACK

SQL on three linked tables



SQL Server

Relational store for the three tables, queried in SSMS.



T-SQL

CTEs, window functions, CASE logic, date functions and joins.



Data model

Orders join Customers on customer_id and Restaurants on restaurant_id.



Validation

Python (pandas) checks of totals, status counts and date range.

Source files: Zomato Customers Data.csv · Zomato Order Data.csv · Zomato Restaurants Data.csv

PROJECT APPROACH

From raw files to recommendations

- | | | |
|-----------|-----------------------------------|--|
| 01 | Load and model the data | Import three CSVs into SQL Server and link orders to customers and restaurants by ID. |
| 02 | Define the metrics | Revenue counts delivered orders; churn means no delivered order in the 90 days before 30 Apr 2026. |
| 03 | Answer 27 questions | Revenue, customers, restaurants, coupons, cancellations and refunds, and churn, each written in T-SQL. |
| 04 | Validate the results | Cross-checked totals, order status counts and the Jan 2024 to Apr 2026 date range against the raw files. |
| 05 | Turn findings into actions | Rank problems by revenue at stake and turn them into recommendations. |

REVENUE ANALYSIS · QUESTION 1 OF 5

What is the total revenue?

SQL QUERY

```
SELECT SUM(order_amount) AS total_revenue  
FROM Orders  
WHERE order_status = 'Delivered';
```

OUTPUT

total_revenue

26859391.4037476

Delivered orders bring in **₹26.86M** in total.

What is the monthly revenue trend?

SQL QUERY

```
SELECT datetrunc(month,order_timestamp)
      AS month,
      sum(order_amount) AS total_revenue
FROM Orders
WHERE order_status = 'Delivered'
GROUP BY datetrunc(month,order_timestamp)
ORDER BY month asc;
```

OUTPUT · FIRST 15 MONTHS

month	total_revenue
2024-01-01	993756.919570923
2024-02-01	914237.788696289
2024-03-01	976562.869888306
2024-04-01	975680.631530762
2024-05-01	959287.189743042
2024-06-01	975701.109466553
2024-07-01	930763.57901001
2024-08-01	984926.560058594

month	total_revenue
2024-09-01	926511.55090332
2024-10-01	1006185.46810913
2024-11-01	969876.559280396
2024-12-01	981302.95954895
2025-01-01	1013576.52134705
2025-02-01	838330.509490967
2025-03-01	1000685.05943298

REVENUE ANALYSIS · QUESTION 3 OF 5

Which city contributes the highest revenue?

SQL QUERY

```
SELECT a.City, sum(order_amount) AS total_revenue
FROM Customers a
JOIN Orders o
ON a.customer_id = o.customer_id
WHERE order_status = 'Delivered'
GROUP BY a.city
ORDER BY SUM(order_amount) desc;
```

OUTPUT

City	total_revenue
Noida	3604414.9690094
Pune	3539648.36965942
Gurugram	3358890.34896851
Hyderabad	3305347.99267578
Bengaluru	3294090.17964172
Delhi	3292014.60681152
Mumbai	3242934.96826172
Chennai	3217081.56872559

REVENUE ANALYSIS · QUESTION 4 OF 5

Which payment mode generates the most revenue?

SQL QUERY

```
SELECT payment_mode, SUM(order_amount) AS total_revenue
FROM Orders
WHERE order_status = 'Delivered'
GROUP BY payment_mode
ORDER BY SUM(order_amount) desc;
```

OUTPUT

payment_mode	total_revenue
UPI	13433746.6039886
Wallet	4514679.88328552
Card	4474070.29698181
Cash	4436894.61949158

REVENUE ANALYSIS · QUESTION 5 OF 5

What is the Average Order Value (AOV)?

SQL QUERY

```
SELECT SUM(order_amount)/count(distinct order_id) AS average_order_value  
FROM Orders  
WHERE order_status = 'Delivered';
```

OUTPUT

average_order_value

900.204156039399

The average delivered order is worth ₹900.

CUSTOMER ANALYSIS · QUESTION 1 OF 4

Who are the top 20 customers by revenue?

SQL QUERY

```
SELECT TOP 20
  customer_id,
  SUM(order_amount - discount_amount + delivery_fee) AS revenue,
  COUNT(order_id) AS delivered_orders
FROM orders
WHERE order_status = 'Delivered'
GROUP BY customer_id
ORDER BY revenue DESC;
```

Who are the top 20 customers by revenue?

OUTPUT · TOP 15 OF 20 ROWS					
customer_id	revenue	delivered_orders	customer_id	revenue	delivered_orders
CO0381	15932.9900741577	16	CO1729	13830.399848938	13
CO0701	15683.5300312042	17	CO2739	13713.0701141357	13
CO3890	15464.4099960327	15	CO2235	13567.3499336243	15
CO1409	14773.0898399353	16	CO4756	13305.5402526855	13
CO1659	14433.2698516846	14	CO1468	13297.9000587463	13
CO2596	14040.830039978	12	CO4598	13291.9299621582	14
CO1957	13936.4500579834	13	CO3054	13251.3399276733	13
CO1795	13853.6899871826	12			

Customer **CO0381** leads at ₹15,933 from 16 delivered orders.

CUSTOMER ANALYSIS · QUESTION 2 OF 4

What percentage of revenue comes from top customers?

SQL QUERY

```
WITH customer_revenue AS (  
  SELECT  
    customer_id,  
    SUM(order_amount - discount_amount  
      + delivery_fee) AS revenue  
  FROM orders  
  WHERE order_status = 'Delivered'  
  GROUP BY customer_id  
) ,  
ranked_customers AS (  
  SELECT  
    customer_id,  
    revenue,  
    ROW_NUMBER() OVER  
      (ORDER BY revenue DESC)  
      AS customer_rank  
  FROM customer_revenue  
)
```

```
SELECT  
  SUM(CASE WHEN customer_rank <= 20  
    THEN revenue ELSE 0 END)  
  AS top_20_revenue,  
  SUM(revenue) AS total_revenue,  
  ROUND(  
    100.0 * SUM(CASE WHEN customer_rank  
      <= 20 THEN revenue ELSE 0 END)  
    / SUM(revenue),  
    2  
  ) AS top_20_revenue_percentage  
FROM ranked_customers;
```

What percentage of revenue comes from top customers?

OUTPUT

top_20_revenue	total_revenue	top_20_revenue_percentage
278248.750200272	26965797.1539078	1.03

The top 20 customers bring in ₹278K of ₹26.97M, only **1.03%** of revenue.

CUSTOMER ANALYSIS · QUESTION 3 OF 4

Which acquisition channel brings the highest-value customers?

SQL QUERY

```
SELECT
  c.Acquisition_channel,
  COUNT(DISTINCT o.customer_id) AS customers,
  SUM(o.order_amount - o.discount_amount + o.delivery_fee) AS total_revenue,
  SUM(o.order_amount - o.discount_amount + o.delivery_fee)
    / COUNT(DISTINCT o.customer_id) AS revenue_per_customer
FROM orders o
JOIN Customers c
  ON o.customer_id = c.Customer_id
WHERE o.order_status = 'Delivered'
GROUP BY c.Acquisition_channel
ORDER BY revenue_per_customer DESC;
```

Which acquisition channel brings the highest-value customers?

OUTPUT

Acquisition_channel	customers	total_revenue	revenue_per_customer
WhatsApp Campaign	989	5432472.02135658	5492.89385374781
Google Ads	990	5406384.09809303	5460.99403847781
Organic	987	5350063.55837631	5420.53045428198
Referral	1000	5364781.18751335	5364.78118751335
Instagram Ads	1024	5406991.48857307	5280.26512555964

WhatsApp Campaign gives the highest revenue per customer at ₹5,493; Instagram Ads the lowest at ₹5,280.

CUSTOMER ANALYSIS · QUESTION 4 OF 4

How many repeat customers do we have?

SQL QUERY

```
WITH frst AS (  
    SELECT  
        customer_id,  
        MIN(order_timestamp) AS first_time  
    FROM Orders  
    GROUP BY customer_id  
)  
SELECT  
    DATETRUNC(month, a.order_timestamp) AS Month,  
    COUNT(DISTINCT a.customer_id) AS repeated_cust  
FROM Orders a  
JOIN frst b  
    ON a.customer_id = b.customer_id  
    AND DATETRUNC(month, a.order_timestamp) <> DATETRUNC(month, b.first_time)  
GROUP BY DATETRUNC(month, a.order_timestamp)  
ORDER BY Month;
```

How many repeat customers do we have?

OUTPUT · FIRST 15 MONTHS

Month	repeated_cust
2024-02-01	432
2024-03-01	773
2024-04-01	1009
2024-05-01	1181
2024-06-01	1254
2024-07-01	1283
2024-08-01	1381
2024-09-01	1416

Month	repeated_cust
2024-10-01	1497
2024-11-01	1448
2024-12-01	1473
2025-01-01	1566
2025-02-01	1385
2025-03-01	1473
2025-04-01	1449

Repeat customers grow from **432** in Feb 2024 to **1,449** in Apr 2025.

RESTAURANT PERFORMANCE · QUESTION 1 OF 5

Which restaurants generate the highest revenue?

SQL QUERY

```
SELECT TOP 10
    r.restaurant_id,
    r.restaurant_name,
    SUM(o.order_amount - o.discount_amount + o.delivery_fee) AS revenue
FROM orders o
JOIN restaurants r
    ON o.restaurant_id = r.restaurant_id
WHERE o.order_status = 'Delivered'
GROUP BY r.restaurant_id, r.restaurant_name
ORDER BY revenue DESC;
```

Which restaurants generate the highest revenue?

OUTPUT

restaurant_id	restaurant_name	revenue
159.00	KFC	166589.759836197
72.00	Biryani By Kilo	160277.99036026
149.00	Barbeque Nation	159827.850112915
193.00	McDonald's	158718.8396492
116.00	Haldiram's	158325.139640808
40.00	Behrouz Biryani	158298.460926056
58.00	KFC	158069.510389328
199.00	Burger King	157824.969747543
192.00	Haldiram's	156491.850166321
62.00	Subway	155758.470384598

RESTAURANT PERFORMANCE · QUESTION 2 OF 5

Which restaurants receive the most orders?

SQL QUERY

```
SELECT TOP 10
    r.restaurant_id,
    r.restaurant_name,
    COUNT(o.order_id) AS total_orders
FROM orders o
JOIN restaurants r
    ON o.restaurant_id = r.restaurant_id
WHERE o.order_status = 'Delivered'
GROUP BY r.restaurant_id, r.restaurant_name
ORDER BY total_orders DESC;
```

Which restaurants receive the most orders?

OUTPUT

restaurant_id	restaurant_name	total_orders
159.00	KFC	185
116.00	Haldiram's	181
72.00	Biryani By Kilo	179
193.00	McDonald's	177
133.00	Burger King	175
82.00	Chaayos	174
58.00	KFC	174
149.00	Barbeque Nation	174
62.00	Subway	173
111.00	Biryani By Kilo	173

RESTAURANT PERFORMANCE · QUESTION 3 OF 5

Which cuisines are most popular?

SQL QUERY

```
SELECT
    r.cuisine,
    COUNT(o.order_id) AS total_orders,
    SUM(o.order_amount - o.discount_amount + o.delivery_fee) AS revenue
FROM orders o
JOIN restaurants r
    ON o.restaurant_id = r.restaurant_id
WHERE o.order_status = 'Delivered'
GROUP BY r.cuisine
ORDER BY total_orders DESC;
```

Which cuisines are most popular?

OUTPUT

cuisine	total_orders	revenue
North Indian	5869	5254133.56032181
Pizza	5706	5161393.74858856
Fast Food	5378	4872601.84784889
Cafe	4536	4117375.39120674
Healthy	4489	4076357.82613564
Biryani	3859	3483934.77980614

North Indian is the most popular cuisine with 5,869 delivered orders; Biryani is the least with 3,859.

RESTAURANT PERFORMANCE · QUESTION 4 OF 5

Do highly-rated restaurants generate more revenue?

SQL QUERY

```
SELECT
  COUNT(DISTINCT r.restaurant_id) AS restaurants,
  r.avg_rating,
  AVG(CAST(x.revenue AS DECIMAL(18,2))) AS avg_revenue
FROM restaurants r
JOIN (
  SELECT
    restaurant_id,
    SUM(order_amount - discount_amount + delivery_fee) AS revenue
  FROM orders
  WHERE order_status = 'Delivered'
  GROUP BY restaurant_id
) x
ON r.restaurant_id = x.restaurant_id
GROUP BY r.avg_rating
ORDER BY r.avg_rating desc;
```

Do highly-rated restaurants generate more revenue?

OUTPUT · 15 RATING BANDS

restaurants	avg_rating	avg_revenue
5	5	138946.402000
13	4.900000009536743	140164.538461
14	4.800000019073486	135402.360714
10	4.69999980926514	131067.615000
18	4.59999990463257	136729.352777
17	4.5	132541.000588
13	4.400000009536743	135398.938461
12	4.300000019073486	128300.195833

restaurants	avg_rating	avg_revenue
17	4.19999980926514	134769.884705
13	4.09999990463257	129845.656923
17	4	137923.265882
7	3.900000009536743	133073.395714
15	3.79999995231628	133886.484000
13	3.700000004768372	138270.165384
9	3.59999990463257	135003.413333

Revenue does not rise with rating: average revenue stays between ₹128K and ₹140K across rating bands.

RESTAURANT PERFORMANCE · QUESTION 5 OF 5

Which are the last 5 restaurants?

SQL QUERY · BASED ON AVERAGE RATINGS

```
SELECT TOP 5
    restaurant_id,
    restaurant_name,
    avg_rating
FROM restaurants
ORDER BY avg_rating ASC;
```

SQL QUERY · BASED ON REVENUE

```
SELECT TOP 5
    r.restaurant_id,
    r.restaurant_name,
    SUM(o.order_amount - o.discount_amount
        + o.delivery_fee) AS revenue,
    r.avg_rating
FROM restaurants r
join orders o
on r.restaurant_id = o.restaurant_id
group by r.restaurant_id,
    r.restaurant_name, r.avg_rating
ORDER BY r.avg_rating asc;
```

Which are the last 5 restaurants?

OUTPUT · BY AVERAGE RATING

restaurant_id	restaurant_name	avg_rating
26.00	Haldiram's	3.5
47.00	Starbucks	3.5
65.00	Burger King	3.5
89.00	Starbucks	3.5
111.00	Biryani By Kilo	3.5

OUTPUT · BY REVENUE

restaurant_id	restaurant_name	revenue	avg_rating
26.00	Haldiram's	228833.899934769	3.5
47.00	Starbucks	212167.410167908	3.5
65.00	Burger King	223368.01076889	3.5
89.00	Starbucks	233790.080192566	3.5
111.00	Biryani By Kilo	245838.890274048	3.5

Five restaurants tie for the lowest rating of **3.5**. The revenue query has no Delivered filter, so its figures count every order status.

COUPON ANALYSIS · QUESTION 1 OF 4

What percentage of orders use coupons?

SQL QUERY

```
SELECT
  COUNT(*) AS total_delivered_orders,
  SUM(CASE WHEN discount_amount > 0 THEN 1 ELSE 0 END) AS coupon_orders,
  ROUND(
    100.0 * SUM(CASE WHEN discount_amount > 0 THEN 1 ELSE 0 END)
    / COUNT(*),
    2
  ) AS coupon_usage_percentage
FROM orders
WHERE order_status = 'Delivered';
```

OUTPUT

total_delivered_orders	coupon_orders	coupon_usage_percentage
29837	12030	40.32000000000000

COUPON ANALYSIS · QUESTION 2 OF 4

Do coupon users spend more than non-coupon users?

SQL QUERY

```
SELECT
  CASE
    WHEN discount_amount > 0 THEN 'Coupon User'
    ELSE 'Non-Coupon User'
  END AS customer_type,
  COUNT(*) AS orders,
  ROUND(AVG(order_amount), 2) AS avg_order_value,
  ROUND(AVG(order_amount - discount_amount
    + delivery_fee), 2) AS avg_realized_revenue
FROM orders
WHERE order_status = 'Delivered'
GROUP BY
  CASE
    WHEN discount_amount > 0 THEN 'Coupon User'
    ELSE 'Non-Coupon User'
  END;
```

OUTPUT

customer_type	orders	avg_order_value	avg_realized_revenue
Coupon User	12030	904.59	854.14
Non-Coupon User	17807	897.24	937.3

COUPON ANALYSIS · QUESTION 3 OF 4

Which city uses the most coupons?

SQL QUERY

```
SELECT
  c.City,
  SUM(CASE WHEN o.discount_amount > 0
    THEN 1 ELSE 0 END) AS coupon_orders
FROM orders o
JOIN customers c
  ON o.customer_id = c.Customer_id
WHERE o.order_status = 'Delivered'
GROUP BY c.City
ORDER BY coupon_orders DESC;
```

OUTPUT

City	coupon_orders
Noida	1600
Pune	1598
Delhi	1507
Gurugram	1483
Bengaluru	1482
Hyderabad	1474
Chennai	1467
Mumbai	1418

COUPON ANALYSIS · QUESTION 4 OF 4

Are coupons helping customer retention?

SQL QUERY

```
WITH customer_orders AS (  
    SELECT  
        customer_id,  
        COUNT(order_id) AS total_orders,  
        SUM(CASE WHEN discount_amount > 0  
            THEN 1 ELSE 0 END) AS coupon_orders  
    FROM orders  
    WHERE order_status = 'Delivered'  
    GROUP BY customer_id  
) ,  
customer_segments AS (  
    SELECT  
        customer_id,  
        total_orders,  
        CASE  
            WHEN coupon_orders > 0 THEN 'Coupon User'  
            ELSE 'Non-Coupon User'  
        END AS customer_type,  
  
        CASE  
            WHEN total_orders > 1 THEN 1  
            ELSE 0  
        END AS is_repeat  
    FROM customer_orders  
)  
SELECT  
    customer_type,  
    COUNT(*) AS customers,  
    SUM(is_repeat) AS repeat_customers,  
    ROUND(  
        100.0 * SUM(is_repeat) / COUNT(*),  
        2  
    ) AS repeat_rate  
FROM customer_segments  
GROUP BY customer_type;
```

Are coupons helping customer retention?

OUTPUT

customer_type	customers	repeat_customers	repeat_rate
Coupon User	4559	4530	99.36000000000000
Non-Coupon User	432	385	89.12000000000000

Coupon users repeat at **99.36%** against 89.12% for non-coupon users, but only 432 customers never used a coupon.

CANCELLATION & REFUND ANALYSIS · QUESTION 1 OF 4

What is the cancellation rate?

SQL QUERY

```
SELECT
  COUNT(*) AS total_orders,
  SUM(CASE WHEN order_status = 'Cancelled' THEN 1 ELSE 0 END) AS cancelled_orders,
  ROUND(
    100.0 * SUM(CASE WHEN order_status = 'Cancelled' THEN 1 ELSE 0 END)
    / COUNT(*),
    2
  ) AS cancellation_rate
FROM orders;
```

OUTPUT

total_orders	cancelled_orders	cancellation_rate
50000	10095	20.19000000000000

CANCELLATION & REFUND ANALYSIS · QUESTION 2 OF 4

What is the refund rate?

SQL QUERY

```
SELECT
  COUNT(*) AS total_orders,
  SUM(CASE WHEN order_status = 'Refunded' THEN 1 ELSE 0 END) AS refunded_orders,
  ROUND(
    100.0 * SUM(CASE WHEN order_status = 'Refunded' THEN 1 ELSE 0 END)
    / COUNT(*),
    2
  ) AS refund_rate
FROM orders;
```

OUTPUT

total_orders	refunded_orders	refund_rate
50000	10068	20.14000000000000

CANCELLATION & REFUND ANALYSIS · QUESTION 3 OF 4

How much revenue is lost due to cancellations?

SQL QUERY

```
SELECT
  SUM(
    order_amount
    - discount_amount
    + delivery_fee
  ) AS lost_revenue
FROM orders
WHERE order_status = 'Cancelled';
```

OUTPUT

lost_revenue

9094421.69070625

Cancelled orders represent **₹9.09M** of lost revenue.

CANCELLATION & REFUND ANALYSIS · QUESTION 4 OF 4

Which restaurants have the highest cancellation rate?

SQL QUERY

```
SELECT TOP 10
    r.restaurant_id,
    r.restaurant_name,
    COUNT(o.order_id) AS total_orders,
    SUM(
        CASE
            WHEN o.order_status = 'Cancelled' THEN 1
            ELSE 0
        END
    ) AS cancelled_orders,
    ROUND(
        100.0 *
        SUM(
            CASE
                WHEN o.order_status = 'Cancelled' THEN 1
                ELSE 0
            END
        ) / COUNT(o.order_id),
        2
    ) AS cancellation_rate
FROM orders o
JOIN restaurants r
    ON o.restaurant_id = r.restaurant_id
GROUP BY
    r.restaurant_id,
    r.restaurant_name
ORDER BY cancellation_rate DESC;
```

Which restaurants have the highest cancellation rate?

OUTPUT

restaurant_id	restaurant_name	total_orders	cancelled_orders	cancellation_rate
194.00	Haldiram's	245	67	27.35000000000000
185.00	Barbeque Nation	251	66	26.29000000000000
120.00	Biryani By Kilo	222	58	26.13000000000000
46.00	KFC	243	63	25.93000000000000
64.00	Chaayos	263	66	25.10000000000000
155.00	McDonald's	267	67	25.09000000000000
49.00	Barbeque Nation	240	60	25.00000000000000
21.00	Barbeque Nation	241	60	24.90000000000000
22.00	Behrouz Biryani	245	61	24.90000000000000
99.00	Chaayos	267	66	24.72000000000000

CUSTOMER CHURN ANALYSIS · QUESTION 1 OF 5

How many customers have churned?

SQL QUERY

```
WITH last_order AS (  
  SELECT  
    customer_id,  
    MAX(order_timestamp) AS last_order_date  
  FROM orders  
  WHERE order_status = 'Delivered'  
  GROUP BY customer_id  
)  
SELECT  
  COUNT(*) AS churned_customers  
FROM customers c  
LEFT JOIN last_order l  
  ON c.Customer_id = l.customer_id  
WHERE l.last_order_date < DATEADD(DAY, -90, '2026-04-30')  
  OR l.last_order_date IS NULL;
```

OUTPUT

churned_customers

2672

CUSTOMER CHURN ANALYSIS · QUESTION 2 OF 5

What is the churn rate?

SQL QUERY

```
WITH last_order AS (  
    SELECT  
        customer_id,  
        MAX(order_timestamp) AS last_order_date  
    FROM orders  
    WHERE order_status = 'Delivered'  
    GROUP BY customer_id  
)  
SELECT  
    COUNT(*) AS total_customers,  
    SUM(  
        CASE  
            WHEN l.last_order_date  
                < DATEADD(DAY, -90, '2026-04-30')  
            OR l.last_order_date IS NULL  
            THEN 1 ELSE 0  
        END  
    ) AS churned_customers,  
  
    ROUND(  
        100.0 * SUM(  
            CASE  
                WHEN l.last_order_date  
                    < DATEADD(DAY, -90, '2026-04-30')  
                OR l.last_order_date IS NULL  
                THEN 1 ELSE 0  
            END  
        ) / COUNT(*),  
        2  
    ) AS churn_rate  
FROM customers c  
LEFT JOIN last_order l  
    ON c.Customer_id = l.customer_id;
```

What is the churn rate?

OUTPUT

total_customers	churned_customers	churn_rate
4999	2672	53.45000000000000

53.45% of customers (2,672 of 4,999) have no delivered order in the 90 days before 30 Apr 2026.

CUSTOMER CHURN ANALYSIS · QUESTION 3 OF 5

Which city has the highest churn?

SQL QUERY

```
WITH last_order AS (  
  SELECT  
    customer_id,  
    MAX(order_timestamp) AS last_order_date  
  FROM orders  
  WHERE order_status = 'Delivered'  
  GROUP BY customer_id  
) ,  
customer_churn AS (  
  SELECT  
    c.Customer_id,  
    c.City,  
    CASE  
      WHEN l.last_order_date  
        < DATEADD(DAY, -90, '2026-04-30')  
      OR l.last_order_date IS NULL  
      THEN 1 ELSE 0  
    END AS churned  
  FROM customers c  
  LEFT JOIN last_order l  
    ON c.Customer_id = l.customer_id  
)  
SELECT  
  City,  
  COUNT(*) AS total_customers,  
  SUM(churned) AS churned_customers,  
  ROUND(100.0 * SUM(churned) / COUNT(*), 2)  
    AS churn_rate  
FROM customer_churn  
GROUP BY City  
ORDER BY churn_rate DESC;
```

Which city has the highest churn?

OUTPUT

City	total_customers	churned_customers	churn_rate
Noida	666	367	55.11000000000000
Bengaluru	619	340	54.93000000000000
Chennai	618	338	54.69000000000000
Pune	644	351	54.50000000000000
Delhi	612	323	52.78000000000000
Gurugram	613	321	52.37000000000000
Mumbai	612	317	51.80000000000000
Hyderabad	615	315	51.22000000000000

CUSTOMER CHURN ANALYSIS · QUESTION 4 OF 5

How much revenue is lost due to churn?

SQL QUERY

```
WITH last_order AS (  
    SELECT  
        customer_id,  
        MAX(order_timestamp) AS last_order_date  
    FROM orders  
    WHERE order_status = 'Delivered'  
    GROUP BY customer_id  
) ,  
churned_customers AS (  
    SELECT  
        c.Customer_id  
    FROM customers c  
    LEFT JOIN last_order l  
        ON c.Customer_id = l.customer_id  
        WHERE l.last_order_date  
            < DATEADD(DAY, -90, '2026-04-30')  
            OR l.last_order_date IS NULL  
)  
SELECT  
    SUM(  
        o.order_amount  
        - o.discount_amount  
        + o.delivery_fee  
    ) AS revenue_from_churned_customers  
FROM orders o  
JOIN churned_customers c  
    ON o.customer_id = c.Customer_id  
WHERE o.order_status = 'Delivered';
```

How much revenue is lost due to churn?

OUTPUT

revenue_from_churned_customers
12975977.6165981

Churned customers account for ₹12.98M of delivered revenue.

CUSTOMER CHURN ANALYSIS · QUESTION 5 OF 5

Who are the high-value churned customers?

SQL QUERY

```
WITH last_order AS (  
    SELECT  
        customer_id,  
        MAX(order_timestamp) AS last_order_date  
    FROM orders  
    WHERE order_status = 'Delivered'  
    GROUP BY customer_id  
) ,  
churned_customers AS (  
    SELECT  
        c.Customer_id,  
        c.Customer_name,  
        c.City,  
        l.last_order_date  
    FROM customers c  
    LEFT JOIN last_order l  
        ON c.Customer_id = l.customer_id  
    WHERE l.last_order_date < DATEADD(DAY, -90,  
        '2026-04-30')  
        OR l.last_order_date IS NULL  
) ,
```

```
customer_revenue AS (  
    SELECT  
        o.customer_id,  
        SUM(  
            o.order_amount  
            - o.discount_amount  
            + o.delivery_fee  
        ) AS lost_revenue,  
        COUNT(o.order_id) AS delivered_orders  
    FROM orders o  
    WHERE o.order_status = 'Delivered'  
    GROUP BY o.customer_id  
)  
SELECT TOP 10  
    c.Customer_id,  
    c.Customer_name,  
    c.City,  
    r.lost_revenue,  
    r.delivered_orders,  
    c.last_order_date  
FROM churned_customers c  
JOIN customer_revenue r  
    ON c.Customer_id = r.customer_id  
ORDER BY r.lost_revenue DESC;
```

Who are the high-value churned customers?

OUTPUT

Customer_id	Customer_name	City	lost_revenue	delivered_orders	last_order_date
CO3054	Rahul Sharma	Noida	13251.3399276733	13	2026-01-22
CO2364	Neha Singh	Noida	13163.420249939	12	2026-01-16
CO4768	Neha Singh	Pune	12768.2999839783	13	2025-11-25
CO1535	Rohan Agarwal	Pune	12484.5099334717	14	2026-01-28
CO3220	Priya Verma	Chennai	12385.7900314331	14	2025-08-19
CO0714	Ananya Mishra	Bengaluru	12378.669921875	14	2026-01-22
CO4585	Priya Verma	Mumbai	12335.0100021362	10	2025-12-27
CO3961	Neha Singh	Noida	12281.8001937866	10	2026-01-17
CO1029	Amit Kumar	Mumbai	12026.5899581909	11	2026-01-21
CO2582	Priya Verma	Delhi	11981.8800048828	11	2026-01-09

KEY INSIGHTS

Revenue is flat and leaking

~₹1M

monthly revenue, flat for 28 months

Delivered revenue stays between ₹0.84M and ₹1.04M, with no growth trend.

50%

of revenue comes through UPI

UPI brings ₹13.4M of ₹26.9M; wallet, card and cash each add about ₹4.5M.

40%

of orders end in cancellation or refund

Only 60% are delivered. Cancellations cost ₹9.1M; the worst restaurants cancel 25–27%.

–₹83

realised revenue per coupon order

40% of delivered orders use a coupon: ₹854 each versus ₹937 without.

53%

of customers have churned

2,672 of 4,999 customers, worth ₹13.0M (48% of revenue). Noida is worst at 55%.

1%

of revenue from the top 20 customers

At 1.03%, revenue is spread widely. Channels and rating bands barely change it.

CONCLUSION

Fix the leaks before buying more growth

Sign-ups are not the problem. About 4 in 10 orders fail and over half of customers have gone quiet, while revenue stays near ₹1M a month.

Cut cancellations and refunds

Audit the 10 highest-cancelling restaurants (25–27%) and set cancellation targets for partners.

Win back high-value churners

Target the 2,672 churned customers with reactivation offers, starting with the top spenders.

Make coupons earn their cost

Coupons cut realised revenue by ₹83 per order, and their link to retention is unproven. Test offers against a control group.

Protect Noida first

Noida earns the most (₹3.6M) and churns the most (55%), so retention work there pays back fastest.